PASTEL: Parasitic Matching-Driven Placement and Routing of Capacitor Arrays With Generalized Ratios in Charge-Redistribution SAR-ADCs

Ye X. Ding, Florin Burcea, Husni Habal, and Helmut E. Graeb, *Fellow, IEEE*

***Abstract*—The charge-redistribution data converter topology is constructed from an array of parallel-connected capacitors with a specific capacitor ratio, and is a fundamental component of successive approximation register (SAR)-analog-to-digital converter (ADC) architectures. This paper presents complementary placement and routing algorithms for such capacitor arrays with the goal of nonlinearity minimization in the target SAR-ADC architecture. Starting from a state-of-the-art static nonlinearity model and its accompanying placement method, generalizations are made for both binary- and nonbinary-weighted capacitor ratios. Taking the generalized nonlinearity model and capacitor routing complexity into consideration, an improved placement method was developed, augmented with a matching-based routing technique driven by routing length adjustment. Analytical results compare the new methodology to the state-of-the-art, achieving superior resolution, linearity, and area.**

***Index Terms*—Capacitor matching, charge-redistribution, layout optimization, nonlinearity minimization, successive approximation register (SAR)-analog-to-digital converter (ADC).**

I. Introduction

THE ANALOG-TO-DIGITAL converter (ADC) is a crucial component in modern integrated circuits. It is responsible for the acquisition of analog signals, followed by their conversion into digital form for further processing. A popular architecture is the successive approximation register (SAR)-ADC, which offers numerous advantages, including low power consumption [1]–[5]. The basic SAR-ADC consists of a signal sample-and-hold circuit, a digital-to-analog converter (DAC) realized with a capacitor array, a voltage comparator, and an SAR with related digital control logic, as depicted in Fig. 1.

This paper targets capacitor array design, which is a primary contributor to the accuracy of the ADC transfer curve. In [1], it was shown that the ADC static performance metrics [integral and differential nonlinearity (INL and DNL)] depend on the variations in the values of the capacitors that form the DAC, as well as suitable matching of these capacitor values, in order to minimize nonlinearity according to equations that are, in turn, specific to the used SAR-ADC architecture. Three principle sources of capacitor variation are recognized.

1) Local random variations modeled statistically (^{sta}).
2) Systematic process biases and gradients (^{sys}).
3) Interconnection parasitic capacitances (^{par}), bearing a systematic influence.

The impact of statistical and systematic (placement-based) variations can be reduced by proper placement; for the former, dispersion, and for the latter, symmetry. Only the parasitic capacitances appearing in parallel with the nominal capacitances affect static linearity. Their effects can be reduced by suitable placement and routing methods to best match interconnect parasitics, which serves as the target topic of this paper.

Typical capacitor unit values lie in the range of ones to tens of fF based on the most recent publications [6]–[8], owing to aggressive trends in technology downscaling and the demand for reduced area and power. The sources of capacitor variation do not scale down by the same factors as the nominal unit values, so the variable component can be large enough to dominate the nominal (intended) value. Careful attention must be paid in order to mitigate performance degradation.

Consequently, a large number of past works concerned with or related to this topic [1], [4], [5], [9]–[13] placed emphasis on the necessity of capacitor matching due to its impact on performance, motivating ongoing research and development.

A linear model was developed in [1] to analytically determine INL and DNL as linear functions of the three aforementioned sources of capacitor variation, and a small nonlinear error term. The final equations for evaluating these metrics are simplified to a simple matrix product of the linear sum of the variations from these three sources.

For the rest of this paper, the following terms shall be used.

1) *(Capacitor) Array:* The DAC formed by $C_0 \cdots C_{M-1}$ as shown in the schematic (Fig. 1).

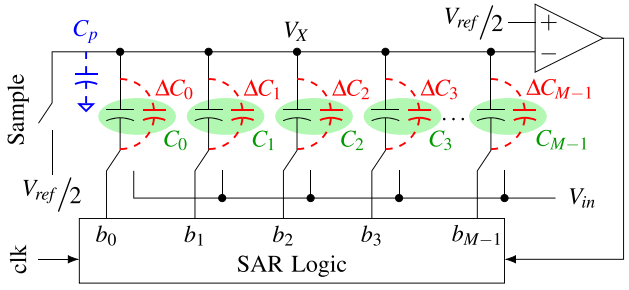


Fig. 1. Schematic of a charge-redistribution SAR-ADC, with its variations: top plate-to-substrate parasitic (blue) and individual capacitor variation (red).

Manuscript received November 20, 2018; revised February 6, 2019; accepted March 24, 2019. Date of publication April 23, 2020; date of current version June 18, 2020. This paper was recommended by Associate Editor S. Mohanty. *(Corresponding author: Florin Burcea.)*

Y. X. Ding and H. Habal are with Infineon Technologies AG, 85579 Neubiberg, Germany.

F. Burcea and H. E. Graeb are with the Chair of Electronic Design Automation, Technical University of Munich, 80333 Munich, Germany (e-mail: florin.burcea@tum.de).

Digital Object Identifier 10.1109/TCAD.2019.2912921

0278-0070 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

2) *Array Capacitor:* Each capacitor in this array.
3) *Unit (Capacitor):* Each of the individual capacitor cells that are connected in parallel to form an array capacitor.
4) *(Capacitor) Matrix:* The resulting grid formed by all unit capacitors after they are placed.
5) *Net*: The electrical connection between the bottom plates of all the unit capacitors of a single array capacitor.
6) *Parasitics:* The parasitic capacitances between net routing (of each capacitor bottom plate) and the top plates.

All row vectors and matrix rows are presented with the most and least significant bits on the left and right, respectively; for column vectors and matrix columns, the most and least significant bits are presented at the top and bottom, respectively.

For each *array capacitor*, the nominal routing problem is to connect its *unit capacitors*' bottom plates with its corresponding exit pin. The problem is extended to also consider routing parasitics and their impact on ADC performance metrics.

This paper is organized as follows. Section II reviews existing literature and outlines the new contributions from this paper. Section III introduces the binary capacitor ratio and extends it for generalized (e.g., nonbinary-weighted) capacitor ratios. Section IV reviews the previously developed nonlinearity model for binary-weighted ratios, then extends it for generalized ratios. Section V introduces a new placement method modified from the method in [1], and depicts its algorithmic implementation. Section VI details the novel routing style designed for the new placement method, and its algorithmic implementation. Analysis and experimental results are presented in Section VII. Section VIII concludes this paper.

II. State-of-the-Art and New Contributions

Optimization of capacitance ratio accuracy in capacitor arrays is a popular research topic. Many papers attempted to achieve this by maximizing the overall correlation between capacitances with respect to an exponential spatial correlation model [1], [5], [13]–[16]. Since capacitance ratio accuracy was the general objective, the resulting solutions could be suboptimal for a different objective function, such as the linearity of a particular SAR-ADC architecture.

In [1], a simple analytical model was derived for a charge-redistribution ADC with binary-weighted capacitor ratios, describing the INL and DNL as linear expressions of capacitor variations. Combining this with the exponential spatial correlation model produced an optimal placement methodology for minimizing the impact of statistical (random fluctuations) and systematic (process gradients) variations on ADC nonlinearity (as measured by INL and DNL). The resulting placement style, named "chessboard," complied with general layout guidelines for this problem [17]: coincidence, symmetry, dispersion, and compactness, with dispersion being the key criterion.

However, the fundamental drawback with the chessboard placement style is that the routing problem was not considered. Interconnection parasitics between the top (common to all array capacitors) and bottom plates (individual to each array capacitor) appear in parallel with the nominal capacitances (depicted by the red-colored parallel capacitors in Fig. 1), additively contributing to their values. Mismatch of these parasitics results in a large mismatch in overall capacitance. Moreover, maximal dispersion implies maximal routing complexity.

The work outlined in [5] considers the spatial correlation model as well, producing a novel placement method. A custom routing technique was not developed, rather one adapted from previous works was used. Furthermore, the resultant INL and DNL were evaluated only for binary-weighted architectures.

In [2], [9], and [10], a strong coupling between the placement and the routing styles can be seen; each was designed with the other in mind. It was concluded that the use of "globally distributed, locally connected" placements provided the best tradeoff between placement-based mismatch and parasitics.

Routing was also considered in [11], where the entire task was formulated as an integer linear programming problem. Finally, Ho *et al.* [12] presented a routing technique by controlling routing lengths in order to precisely maintain the capacitance ratio. Inter-routing coupling was also considered, based on the distances between routing. In both [11] and [12], the goals were limited to reduce variation in general, without being directed toward metric (e.g., nonlinearity) improvement.

In all of [2], [5], and [10]–[12], the results of evaluating INL and DNL were presented; however, complete definitions were used and not with the linearized, simple model from [1].

This paper provides the following contributions.

1) A novel hybrid placement algorithm to place capacitor arrays with generalized weightings is presented. The algorithm considers the tradeoff between capacitor dispersion and routability not considered in the chessboard placement algorithm described in [1].
2) A novel routing method to complement the proposed hybrid placement is introduced. Routing lengths (and by extension capacitances) are iteratively adjusted to reach the overall goal of minimizing ADC INL and DNL. This method is inspired by Ho *et al.* [12], where a complex optimization problem is presented to precisely match capacitor ratio values. However, in this paper, the algorithm is different and more simplified (wire length adjustment). Moreover, the optimization goal is different (to minimize overall INL and DNL), achieved using a different routing solution.
3) A linearized analytical model of INL and DNL for any SAR-ADC with a generalized capacitor ratio is defined. The model is then used to evaluate three sources of nonlinearity independently, as well as in connection with each other: statistical, systematic, and parasitic. The analytical model guides the resultant layout automation algorithm, named "PASTEL," toward minimization of INL and DNL to desirably low levels, by consideration of: a) capacitor variation caused by placement-related mismatch (due to placement dispersion) and b) the components caused by interconnection parasitics (due to routing).

III. Generalized Capacitor Ratio and Redundancy in ADCs

The SAR-ADC includes a DAC comprised of a capacitor array, as per Fig. 1. Let $\boldsymbol{c}$ be the ordered array of M DAC capacitors, and C_T be the total sum of capacitor values; with respect to Fig. 1, each effective array capacitor C_j (shaded in green) is the parallel combination (and hence, sum) of a nominal capacitance (black) and its associated variation (red)

$$\boldsymbol{c}^{\mathsf{T}} = [C_{M-1}, C_{M-2}, \ldots, C_1, C_0], C_T = \sum_{j=0}^{M-1} C_j. \quad (1)$$

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

At any step in the conversion process, the voltage V_X at the DAC output (the capacitor top plate) is compared to $V_{\text{ref}}/2$ [18]. Initially, $V_X = V_{\text{ref}}/2$ is sampled on the top plate, and the input voltage, V_{in}, is sampled on the bottom plates of C_{M-1} to C_0. The total charge introduced into the array is

$$Q = \left(\frac{V_{\text{ref}}}{2} - V_{\text{in}}\right) \cdot C_T + \frac{V_{\text{ref}}}{2} \cdot C_p \tag{2}$$

where C_p is the top plate-to-substrate parasitic from Fig. 1.

Throughout the conversion, the bottom plates are driven by the SAR logic and thus connected to either V_{ref} or ground. Let the bit vector $\boldsymbol{b}$ denote the ordered M bottom-plate connections in Fig. 1; 1 and 0 represent V_{ref} and VSS, respectively

$$\boldsymbol{b}^{\mathsf{T}} = [b_{M-1}, b_{M-2}, \ldots, b_1, b_0], \;\; b_j \in \{0, 1\}. \tag{3}$$

After sampling, the top plate is disconnected and remains floating. Thus, the total charge stays constant, expressed as

$$\begin{aligned} Q &= \sum_{j=0}^{M-1} \left(V_X \cdot C_j - V_{\text{ref}} \cdot b_j \cdot C_j\right) + V_X \cdot C_p \\ &\overset{(1),(3)}{=} V_X \cdot C_T + V_X \cdot C_p - V_{\text{ref}} \cdot \overbrace{\boldsymbol{b}^{\mathsf{T}} \cdot \boldsymbol{c}}^{\sum_{j=0}^{M-1} b_j \cdot C_j}. \end{aligned} \tag{4}$$

The total charge, Q, remains constant between (2) and (4). Equating the expressions and rearranging the terms gives

$$V_{\text{in}} = \left(\frac{V_{\text{ref}}}{2} - V_X\right) \cdot \left(1 + \frac{C_p}{C_T}\right) + V_{\text{ref}} \cdot \left(\frac{\boldsymbol{b}^{\mathsf{T}} \cdot \boldsymbol{c}}{C_T}\right). \tag{5}$$

From (5), a comparison of $V_{\text{ref}}/2$ with V_X is equivalent to a comparison of V_{in} and $V_{\text{ref}} \cdot \boldsymbol{b}^{\mathsf{T}} \cdot \boldsymbol{c} \; / C_T$

$$V_{\text{ref}}/2 > V_X \iff V_{\text{in}} > V_{\text{ref}} \cdot \boldsymbol{b}^{\mathsf{T}} \cdot \boldsymbol{c} \; / C_T = V(\boldsymbol{b}). \tag{6}$$

$V(\boldsymbol{b})$ in (6) defines the current state of the ADC. The SAR logic output, $\boldsymbol{b}$, can be an intermediate or the final conversion result. To distinguish the final result, an index i is used, so that $\boldsymbol{b}(i)$ is the binary representation of i. $V(\boldsymbol{b}(i))$ then denotes the voltage at which the ADC transfer curve (discrete output level) changes from level $i-1$ to i. This is the ADC counterpart to the output voltage of a DAC and hence bears the same expression, but without a top-plate parasitic capacitance (C_p) term.

Although present in (2) and (4), C_p does not appear in the expression of $V(\boldsymbol{b}(i))$. This parasitic capacitance does not influence the conversion thresholds, because its bottom plate is connected to the (always grounded) substrate, and the top-plate voltage V_X converges to the initial value, $V_{\text{ref}}/2$ [19].

Equation (6) holds irrespective of any variation source, modeled as a part of $\boldsymbol{c}$. Ideal voltages are obtained by using nominal capacitances, replacing $\boldsymbol{c}$ with $\boldsymbol{c}_0$, and C_T with C_{T0}, such that

$$\boldsymbol{c} = \boldsymbol{c}_0 + \boldsymbol{\Delta c}, \;\; \boldsymbol{\Delta c}^{\mathsf{T}} = [\Delta C_{M-1}, \Delta C_{M-2}, \ldots, \Delta C_0] \tag{7}$$

$$C_T = C_{T0} + \Delta C_T, \;\; \Delta C_T = \sum_{j=0}^{M-1} \Delta C_j. \tag{8}$$

$\boldsymbol{c}_0$ depends on the architecture and shall be defined later.

A. Binary-Weighted Capacitor Ratios

In the binary-weighted case, there are $M = N + 1$ array capacitors, where N of them are ordered in increasing powers of 2, such that the capacitor indices range from 0 to N. Taking C_u to represent the fixed unit capacitance, the capacitances are equal to $C_u \cdot 2^{j-1}$, $1 \le j \le N$, with one additional capacitor with a grounded bottom plate forming the nominal device of $\hat{C}_0$. Note that symbols specific to the binary ratio bear a hat ($\hat{}$) above them, so as to distinguish them as a special case.

This grounded capacitor is introduced so that the total sum of the capacitances is $C_{T0} = C_u \cdot 2^N$, allowing a normalized resolution of $(1/2^N)$ instead of $[1/(2^N - 1)]$. It also fixes $\hat{b}_0$ at 0, so $\hat{b}_{M-1}\hat{b}_{M-2}\cdots\hat{b}_1$ is the binary representation for the integer i, $0 \le i \le 2^N - 1$. The INL is defined for this range, but the DNL excludes $i = 0$. The vector representations of the binary ratio ($\hat{\boldsymbol{n}}$) and the (nominal) array capacitor values ($\hat{\boldsymbol{c}}_0$) are

$$\hat{\boldsymbol{n}}^{\mathsf{T}} = [\hat{n}_{M-1}, \hat{n}_{M-2}, \ldots, \hat{n}_1, \hat{n}_0] = [2^{N-1}, \ldots, 2, 1, 1] \tag{9}$$

$$\hat{\boldsymbol{c}}_0^{\mathsf{T}} = C_u \cdot \hat{\boldsymbol{n}}^{\mathsf{T}} \overset{(9)}{=} C_u \cdot [2^{N-1}, \ldots, 2, 1, 1]. \tag{10}$$

Note that $\hat{\boldsymbol{n}}$ includes the multiplicities of all capacitors, because the input voltage is sampled on all capacitors.

B. Generalized Capacitor Ratios

For generalized ratios, there are M array capacitors, more than the number of bits N. An equivalent vector $\boldsymbol{n}$ can also be defined in the same way as (9), and consequently the nominal array capacitances would be $C_u \cdot n_j$, $0 \le j \le M - 1$. Since the sum of the multiplicities is still 2^N, the resolution V_{LSB} is $V_{\text{ref}}/2^N$, and (10) is generalized to

$$\boldsymbol{c}_0^{\mathsf{T}} \overset{(10)}{=} C_u \cdot \boldsymbol{n}^{\mathsf{T}} = C_u \cdot [n_{M-1}, n_{M-2}, \ldots, n_1, n_0]. \tag{11}$$

The specific generalized capacitor ratio in the 12-bit case study used for the purposes of results in this paper is

$$\boldsymbol{n}^{\mathsf{T}} = [1868, 1012, 552, 303, 168, 90, 48, 26, 14, 8, 4, 2, 1]. \tag{12}$$

Definitions, such as (10), can be extended to their generalized forms. Consider, for example, a 4-bit ADC ($N = 4$). With binary-weighted ratios, it would consist of capacitors $\hat{C}_4$ to $\hat{C}_0$, with $\hat{\boldsymbol{n}} = [8, 4, 2, 1, 1]$. Each integer between 0 and $2^4 - 1$ can be uniquely represented as a sum of the capacitor multiplicities (excluding the rightmost element, corresponding to $\hat{C}_0$). For example, $13 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1$. In other words, 1101 is the representation of the integer 13. Considering the iterative conversion process, the conversion result, in a 4-bit unsigned binary representation, is obtained after four iterations.

However, a 4-bit ADC can also be realized with a (nonbinary) ratio $\boldsymbol{n} = [8, 3, 2, 2, 1]$. In this case, certain integers have multiple representations as a sum of the capacitor multiplicities (here, all capacitors between C_4 and C_0 are considered in the sum). Hence, 13 can be written as $1{\cdot}8{+}1{\cdot}3{+}1{\cdot}2{+}0{\cdot}2{+}0{\cdot}1$, as $1{\cdot}8{+}1{\cdot}3{+}0{\cdot}2{+}1{\cdot}2{+}0{\cdot}1$, or as $1{\cdot}8{+}0{\cdot}3{+}1{\cdot}2{+}1{\cdot}2{+}1{\cdot}1$. In other words, the integer 13 can be represented by either 11100, 11010, or 10111. The conversion process is still iterative, and so the conversion result is a 5-bit value obtained after five iterations. This takes one more iteration than the binary case, as well as an additional bit for representing integers in the same range, 0 and $2^4 - 1$. The bit weighting is certainly no longer binary. For example, the second bit from the left has a

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
OUTPUT LEVELS FOR A NONBINARY ADC WITH $\boldsymbol{n} = [8, 3, 2, 2, 1]^{\mathsf{T}}$

Intermediate Pattern				Output	i_x	Threshold
			1111□	**11111**	$\mathbf{16}_1$	
		111□□		**11110**	$\mathbf{15}_1$	**16**
			1110□	**11101**	$\mathbf{14}_2$	**15**
	11□□□			**11100**	$\mathbf{13}_3$	**14**
			1101□	11011	14_1	
		110□□		11010	13_2	14 **13**
			1100□	**11001**	$\mathbf{12}_2$	
1□□□□				**11000**	$\mathbf{11}_3$	**12** **11**
			1011□	10111	13_1	
		101□□		10110	12_1	13 12
			1010□	10101	11_2	
	10□□□			**10100**	$\mathbf{10}_2$	**11** **10**
			1001□	10011	11_1	
		100□□		10010	10_1	11 **10**
			1000□	**10001**	$\mathbf{9}_1$	
				10000	$\mathbf{8}_2$	**9** **8**
			0111□	01111	8_1	
		011□□		**01110**	$\mathbf{7}_1$	**8** **7**
			0110□	**01101**	$\mathbf{6}_2$	
	01□□□			**01100**	$\mathbf{5}_3$	**6** **5**
			0101□	01011	6_1	
		010□□		01010	5_2	6 **5**
			0100□	**01001**	$\mathbf{4}_2$	
0□□□□				**01000**	$\mathbf{3}_3$	**4** **3**
			0011□	00111	5_1	
		001□□		00110	4_1	5 4
			0010□	00101	3_2	
	00□□□			**00100**	$\mathbf{2}_2$	**3** **2**
			0001□	00011	3_1	
		000□□		00010	2_1	3 **2**
			0000□	**00001**	$\mathbf{1}_1$	
				00000	$\mathbf{0}_1$	**1**

weight of 3 instead of 4. To map the conversion results 11100, 11010, and 10111 to 1011 (the common binary form of 13), additional digital logic for decoding is needed.

Table I depicts the intermediate and final bit patterns, as well as the comparison thresholds (representing voltages normalized to the resolution V_{LSB}) for this 4-bit ADC example. If each comparator decision were fault-free (and the capacitor ratio had nominal values without variations, to be discussed in Section IV), the final bit pattern would be one of the entries in bold in the *Output* column of Table I, with the corresponding value of i in the next column. These patterns shall be referred to as the *nominal* patterns. Out of several conversion results corresponding to one integer, the nominal one is the uppermost occurrence in the table. Since the conversion starts from the most significant bit, this bit is the first to be recorded as part of the final result. For the input $13\,V_{\mathrm{LSB}} < V_{\mathrm{in}} < 14\,V_{\mathrm{LSB}}$, for example, its result of 13 bears the nominal representation 11100, not 11010, nor 10111.

The equivalent of Table I for the 4-bit binary-weighted case is constructed in the same way. For the *intermediate pattern* columns, since there are now N bits rather than M, and considering $N = M - 1$ in this example, they are the same as either the top or bottom halves of the table, but without the most significant bit. The i values represent each integer from 0 to 15 ($2^N - 1$) without duplicates, and the corresponding *threshold* column values range from 1 to (also) $2^N - 1$.

The following example demonstrates the benefit of redundancy. Table II shows all conversion steps for two cases: 1) when all comparator decisions are correct (nominal behavior) and 2) when one decision is erroneous. In the first case, the nominal result, 11100, is obtained. In the second case, a wrong comparator decision is taken in step 2, such that the

TABLE II
COMPARISON OF THE CONVERSION PROCESS OF INPUT i USING TABLE I, WHERE $13 < i < 14$, WITHOUT AND WITH A BIT ERROR IN THE PROCESS

Step	Nominal Behavior			Behavior due to a Bit Error		
	Decision	Result	Pattern	Decision	Result	Pattern
1	$i > 8$	True	1□□□□	$i > 8$	True	1□□□□
2	$i > 11$	True	11□□□	$i > 11$	False	10□□□
3	$i > 13$	True	111□□	$i > 10$	True	101□□
4	$i > 15$	False	1110□	$i > 12$	True	1011□
5	$i > 14$	False	11100 (13)	$i > 13$	True	10111 (13)

second bit is 0 instead of (the correct) 1. From here, the process follows a different branch of the decision tree, compared to the nominal case. This conversion process continues with no further errors, producing 10111, which also corresponds to 13. Due to the redundancy, the final result is correct, despite the error. It can be concluded that from an algorithmic perspective, the use of redundancy lowers efficiency.

A high-frequency ADC demands a fast and precise comparator, but such a comparator is power-hungry, yet power efficiency is a fundamental requirement in modern designs. Therefore, it is more convenient to use a slower and less precise comparator, which also reduces power consumption. The erroneous decisions of this comparator are compensated by the redundancy in the conversion algorithm. Power reduction is thus paid for by additional conversion cycle(s) [5], [20].

Interestingly, SAR-ADCs with redundancy can actually operate at higher frequencies than the binary variant, as the cycle period can be decreased by purposely permitting *incomplete* voltage settling per cycle [3], [20]. This is possible due to their ability to recover from bit errors in the conversion process. Furthermore, because the algorithm implicitly corrects any comparator offset, further analog calibration is not needed [21]. Nevertheless, the sum of the capacitor multiplicities (the total numbers of units) stays the same (2^N) for the same number of bits (N), in order to maintain the same resolution. Consequently, total capacitor matrix area does not increase.

The advantages above are related to ADC dynamic behavior. However, the scope of this paper is to study the ADC static linearity. The previous discussion was intended to justify the importance of these architectures due to the advantages of redundancy, and to review the principles of ADC architectures with generalized capacitor ratios, as a prelude to the derivation of an analytical reduced-order model in Section IV-B.

IV. ANALYTICAL REDUCED-ORDER MODEL OF NONLINEARITY ERRORS

A. Binary-Weighted Architectures

The main source of static nonlinearity for an ADC's transfer curve lies in the capacitor ratio's inaccuracies.

In [1], an analytical model was derived, describing the INL and DNL of a charge-redistribution DAC as linear expressions of the capacitor variations. Applying the derivation from [1], the INL and DNL for a binary-weighted charge-redistribution ADC can be written in terms of the capacitor variations $\boldsymbol{\Delta c}$

$$\mathrm{INL}(i) = (1 + \Delta C_T/C_{T0})^{-1} \cdot \overline{\mathrm{INL}}(i) \tag{13}$$

$$\overline{\mathrm{INL}}(i) = \boldsymbol{b}(i)^{\mathsf{T}} \cdot (\boldsymbol{I}_{N+1} - 2^{-N} \cdot \boldsymbol{n} \cdot \mathbf{1}_{N+1}^{\mathsf{T}}) \cdot \boldsymbol{\Delta c}/C_u \tag{14}$$

$$\mathrm{DNL}(i) = (1 + \Delta C_T/C_{T0})^{-1} \cdot \overline{\mathrm{DNL}}(i) \tag{15}$$

$$\overline{\mathrm{DNL}}(i) = (\boldsymbol{\Delta b}(i)^{\mathsf{T}} - 2^{-N} \cdot \mathbf{1}_{N+1}^{\mathsf{T}}) \cdot \boldsymbol{\Delta c}/C_u. \tag{16}$$

$\mathbf{1}_x$ is a column vector of x 1s and $\boldsymbol{I}_x$ is the $x \times x$ identity matrix.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

INL(i) and DNL(i) were written in this form in order to separate them into a product of a nonlinear term $(1 + \Delta C_T/C_{T0})^{-1}$, and a linear expression with respect to capacitor variations, $\mathbf{\Delta c}$. If capacitor variations are small, then the nonlinear term can be neglected [1], referred to as the *linearity assumption* from here on. This linearity assumption must be checked with the presentation of any analysis results.

For each i, $\overline{\text{INL}}(i)$ is different, because it depends on $\boldsymbol{b}(i)$, which is unique for each i. There are $2^N - 1$ unique $\overline{\text{INL}}$ components in total. Defining $\boldsymbol{B} = [\boldsymbol{b}(1); \boldsymbol{b}(2); \ldots; \boldsymbol{b}(2^N - 1)]$ sets up the vector form of $\overline{\text{INL}}$

$$\overline{\mathbf{INL}} = \boldsymbol{B}^\mathsf{T} \cdot \left(\boldsymbol{I}_{N+1} - 2^{-N} \cdot \boldsymbol{n} \cdot \mathbf{1}_{N+1}^\mathsf{T}\right) \cdot \mathbf{\Delta c}/C_u. \tag{17}$$

The $\overline{\text{DNL}}$ is based on the difference between a code and its preceding code, i.e., a transition. $\mathbf{\Delta b}(i)$ of (16) is defined as a *difference pattern*, representing the bit changes that occur during a transition from $i-1$ to i, that is (for the binary case)

$$\mathbf{\Delta}\hat{\boldsymbol{b}}(i) = \boldsymbol{b}(i) - \boldsymbol{b}(i-1), \;\; 1 \le i \le 2^N - 1. \tag{18}$$

In [1], it was shown that for an N-bit binary-weighted architecture there are only N unique difference patterns $\mathbf{\Delta b}(i)$, denoted as $\mathbf{\Delta}\hat{\boldsymbol{b}}_k$, $1 \le k \le N$, grouped in the matrix $\mathbf{\Delta}\hat{\boldsymbol{B}}$

$$\mathbf{\Delta}\hat{\boldsymbol{B}}^\mathsf{T} = \begin{bmatrix} \mathbf{\Delta}\hat{\boldsymbol{b}}_1^\mathsf{T} \\ \mathbf{\Delta}\hat{\boldsymbol{b}}_2^\mathsf{T} \\ \vdots \\ \mathbf{\Delta}\hat{\boldsymbol{b}}_{N-1}^\mathsf{T} \\ \mathbf{\Delta}\hat{\boldsymbol{b}}_N^\mathsf{T} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & \cdots & 0 & 1 & -1 & 0 \\ & & & \vdots & & & & \\ 0 & 1 & -1 & \cdots & -1 & -1 & -1 & 0 \\ 1 & -1 & -1 & \cdots & -1 & -1 & -1 & 0 \end{bmatrix}. \tag{19}$$

Consequently, there are only N unique $\overline{\text{DNL}}$ expressions, denoted as $\overline{\text{DNL}}_k$, $1 \le k \le N$, grouped in the vector $\overline{\mathbf{DNL}}$

$$\overline{\text{DNL}}_k = \left(\mathbf{\Delta}\hat{\boldsymbol{b}}_k^\mathsf{T} - 2^{-N} \cdot \mathbf{1}_{N+1}^\mathsf{T}\right) \cdot \mathbf{\Delta c}/C_u \tag{20}$$

$$\overline{\mathbf{DNL}} = \left(\mathbf{\Delta}\hat{\boldsymbol{B}}^\mathsf{T} - 2^{-N} \cdot \mathbf{1}_N \cdot \mathbf{1}_{N+1}^\mathsf{T}\right) \cdot \mathbf{\Delta c}/C_u. \tag{21}$$

B. Architectures With Generalized Capacitor Ratios

A similar linearized analytical model with respect to capacitor variations can be derived for ADCs with nonbinary capacitor ratios. Consider an N-bit architecture with M capacitors. In the sampling phase, the situation is unchanged. This leads again to (2), where the vector $\boldsymbol{b}(i)$, defined in (3), describes the bottom-plate connections. Now, however, $M-1$ does not necessarily equal N, b_0 is no longer tied to 0, and $\boldsymbol{b}(i)$ relates to i differently: $i = \boldsymbol{b}(i)^\mathsf{T} \cdot \boldsymbol{n}$. The binary representation of i is $b_{M-1}b_{M-2}\cdots b_1 b_0$ (including b_0). Nevertheless, at any conversion step (4), and consequently (6) hold, as they are independent of the bit pattern and capacitor values.

The different mapping $\boldsymbol{b}(i) \mapsto i$ causes the major differences between the binary and nonbinary architectures. They will be explained using the 4-bit example from Section III-B.

1) Bit Patterns: Given multiple $\boldsymbol{b}(i)$ representations for i, let $p(i)$ denote the *number* of possible representations for each i. Generalizing $\boldsymbol{b}(i)$ from (3), $\boldsymbol{b}_x(i)$, $1 \le x \le p(i)$ denotes the indexed representations of i, for example

$$\boldsymbol{b}_1(2) = [0,0,1,0,0]^\mathsf{T}, \;\; \boldsymbol{b}_2(2) = [0,0,0,1,0]^\mathsf{T}, \;\; p(2) = 2. \tag{22}$$

2) Static Patterns: The linearized model presented refers only to the static behavior, i.e., it is assumed that all comparator decisions are correct. Under this assumption, it can be shown that certain bit patterns cannot occur. If $\boldsymbol{b}(i_1) > \boldsymbol{b}(i_2)$, but $i_2 > i_1$, then the pattern $\boldsymbol{b}(i_2)$ *cannot* occur under static conditions and/or in the presence of small variations. An example is $\boldsymbol{b}(13) = [1,0,1,1,1]^\mathsf{T}$, since $\boldsymbol{b}(13) = [1,0,1,1,1] < [1,1,0,0,1] = \boldsymbol{b}(12)$, as shown in Table I. This pattern can occur only when the conversion process is affected by dynamic errors; such a case is illustrated in the right-hand side of Table II. Hence, the pattern $[0,0,0,1,1]$ for $i = 3$ is not allowed. $p'(i)$, $p'(i) \le p(i)$ is defined to denote only the number of patterns *valid in static conditions* (depicted in bold in the *threshold* column of Table I), for example

$$\boldsymbol{b}_1(3) = [0,1,0,0,0]^\mathsf{T}, \;\; \boldsymbol{b}_2(3) = [0,0,1,0,1]^\mathsf{T}, \;\; p'(3) = 2. \tag{23}$$

3) Transition Voltages: It can be shown that from multiple bit patterns for i, only one will determine (under static conditions) the transition voltage from $i-1$ to i, namely the pattern with the smallest $\boldsymbol{b}(i)^\mathsf{T} \cdot \boldsymbol{c}/C_u$. Consider the capacitor ratio $\boldsymbol{c}/C_u = [7.8, 3.1, 2.1, 1.9, 1]^\mathsf{T}$. Then, $\boldsymbol{b}_1(2)^\mathsf{T} \cdot \boldsymbol{c}/C_u = 2.1$, and $\boldsymbol{b}_2(2)^\mathsf{T} \cdot \boldsymbol{c}/C_u = 1.9$. Therefore, $\boldsymbol{b}_2(2)$ will characterize the transition of $i-1$ to i. If $\boldsymbol{c}/C_u = [7.8, 3.1, 1.9, 2.1, 1]$, then $\boldsymbol{b}_1(2)^\mathsf{T} \cdot \boldsymbol{c}/C_u = 1.9$, $\boldsymbol{b}_2(2)^\mathsf{T} \cdot \boldsymbol{c}/C_u = 2.1$, and $\boldsymbol{b}_1(2)$ will characterize the transition.

Since the minimum is not a linear function, the approach of conservatively considering all possible transitions shall be taken, in order to retain linearity. We define α as the number of valid static patterns that determine the transition voltages

$$\alpha = \sum_{i=1}^{2^N} p'(i). \tag{24}$$

For the 4-bit example in Table I, α is 23; for the 12-bit architecture described by $\boldsymbol{n}$ in (12), α is 4459. All transition voltages $V(\boldsymbol{b}_k)$, $1 \le k \le \alpha$, can be obtained with (6), where $\boldsymbol{b}_k$ represents all of the static patterns. Because $V(\boldsymbol{b}_k)$ has the same form as in the binary-weighted case, the INL components are calculated similarly, obtaining (13), with the linear part also taking on the form of (14)

$$\overline{\text{INL}}_k = \boldsymbol{b}_k^\mathsf{T} \cdot (\boldsymbol{I}_M - 2^{-N} \cdot \boldsymbol{n} \cdot \mathbf{1}_M^\mathsf{T}) \cdot \mathbf{\Delta c}/C_u, \;\; 1 \le k \le \alpha. \tag{25}$$

Vectorizing all values of $\boldsymbol{b}_k$ as the matrix $\boldsymbol{B}$, the full set of $\overline{\text{INL}}_k$ equations is hence

$$\overline{\mathbf{INL}} = \boldsymbol{B}^\mathsf{T} \cdot (\boldsymbol{I}_M - 2^{-N} \cdot \boldsymbol{n} \cdot \mathbf{1}_M^\mathsf{T}) \cdot \mathbf{\Delta c}/C_u. \tag{26}$$

The number of components has increased from $2^N - 1$ to α, because of the conservative approach of considering all possible transitions. Also, the $\boldsymbol{b}_k$ vectors are different from the $\boldsymbol{b}(i)$ vectors from (14) because of the differing architectures.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

4) Difference Patterns: Difference patterns are defined as the element-wise subtraction of two (consecutive) transition patterns. With the existence of multiple possible representations of i and having identified the transition patterns, the difference patterns can be obtained, for calculating the DNL. Each possible representation of i shall be denoted as i_x (as per the corresponding column in Table I), enumerating the occurrences of multiple patterns for the same value of i. For the i corresponding to the *nominal* output pattern (in bold font), its value of x, is equal to $p(i)$.

For $1 \le i \le 2^N - 1$, the DNL(i) is calculated based on $V(\boldsymbol{b}(i))$ and $V(\boldsymbol{b}(i-1))$. However, i and $i-1$ could have multiple representations, so they are enumerated using y and $z : i_y$, $1 \le y \le p'(i)$, and $(i-1)_z$, $1 \le z \le p'(i-1)$. The conservative approach of considering all possible combinations will be taken, in order to retain linearity. Therefore, starting from the original definition of the DNL, the equivalent of DNL(i) becomes DNL(i_y, $(i-1)_z$)

$$\mathrm{DNL}(i_y, (i-1)_z) = \frac{\left[V(\boldsymbol{b}_y(i)) - V(\boldsymbol{b}_z(i-1))\right] - V_{\mathrm{LSB}}}{V_{\mathrm{LSB}}}. \quad (27)$$

As per the binary-weighted architecture, the expression of DNL(i_y, $(i-1)_z$) can be linearized, obtaining a result similar to (16), in which $\boldsymbol{\Delta b}(i)$ is replaced by $\boldsymbol{\Delta b}(i_y, (i-1)_z)$

$$\overline{\mathrm{DNL}}(i_y, (i-1)_z) = (\boldsymbol{\Delta b}(i_y, (i-1)_z)^{\mathsf{T}} - 2^{-N} \cdot \mathbf{1}_M^{\mathsf{T}}) \cdot \boldsymbol{\Delta c}/C_u \quad (28)$$

$$\boldsymbol{\Delta b}(i_y, (i-1)_z) = \boldsymbol{b}_y(i) - \boldsymbol{b}_z(i-1),\ 1 \le i \le 2^N - 1. \quad (29)$$

For example, in the case, where $i = 3$, there are two different static patterns for both $i = 3$, and $i - 1 = 2$, giving $2 \times 2 = 4$ possible transitions: $(3_1, 2_1)$, $(3_2, 2_1)$, $(3_1, 2_2)$, and $(3_2, 2_2)$.

The size of a single difference pattern $\boldsymbol{\Delta b}(i_y, (i-1)_z)$ is $1 \times M$, corresponding to the number of capacitors in the array. We define β as the total number of such difference patterns

$$\beta = \sum_{i=1}^{2^N} p'(i) \cdot p'(i-1). \quad (30)$$

Continuing the examples, the 4-bit example has a β of 32. Its difference patterns are

$$\begin{aligned}
\boldsymbol{\Delta b}(1,0)^{\mathsf{T}} &= [0,\ 0,\ 0,\ 0,\ 1]\\
\boldsymbol{\Delta b}(2_1,1)^{\mathsf{T}} &= [0,\ 0,\ 0,\ 1,\ -1]\\
\boldsymbol{\Delta b}(2_2,1)^{\mathsf{T}} &= [0,\ 0,\ 1,\ 0,\ -1]\\
&\cdots\\
\boldsymbol{\Delta b}(14,13_2)^{\mathsf{T}} &= [0,\ 0,\ 0,\ 0,\ 1]\\
\boldsymbol{\Delta b}(15,14)^{\mathsf{T}} &= [0,\ 0,\ 0,\ 1,\ -1]\\
\boldsymbol{\Delta b}(16,15)^{\mathsf{T}} &= [0,\ 0,\ 0,\ 0,\ 1]
\end{aligned} \quad (31)$$

$\boldsymbol{n}$ from (12) has a β of 4823.

5) Unique Difference Patterns: Just like in the binary-weighted case, identical difference patterns exist. For example, it can be seen that $\boldsymbol{\Delta b}(1,0) = \boldsymbol{\Delta b}(14,13_2) = \boldsymbol{\Delta b}(16,15)$ and $\boldsymbol{\Delta b}(2_1,1) = \boldsymbol{\Delta b}(15,14)$.

Thus, it is enough to consider only the unique patterns, greatly reducing the number of DNL components. The unique patterns shall be enumerated using k as $\boldsymbol{\Delta b}_k$, so $\boldsymbol{\Delta B}$ vectorizes the set of $\boldsymbol{\Delta b}_k$. In the case of generalized capacitor ratios, $\boldsymbol{\Delta B}$ no longer has a regular pattern, and must be calculated with programmatic aid. The number of unique difference patterns is now more than the number of bits. For the 4-bit example from Table I, there are seven unique patterns, giving $\boldsymbol{\Delta B}$

$$\boldsymbol{\Delta B}^{\mathsf{T}} = \begin{bmatrix} \boldsymbol{\Delta b}_1^{\mathsf{T}} \\ \boldsymbol{\Delta b}_2^{\mathsf{T}} \\ \boldsymbol{\Delta b}_3^{\mathsf{T}} \\ \boldsymbol{\Delta b}_4^{\mathsf{T}} \\ \boldsymbol{\Delta b}_5^{\mathsf{T}} \\ \boldsymbol{\Delta b}_6^{\mathsf{T}} \\ \boldsymbol{\Delta b}_7^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & -1 & 1 \\ 0 & 0 & 1 & 0 & -1 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 \\ 1 & -1 & -1 & -1 & 0 \end{bmatrix}. \quad (32)$$

For $\boldsymbol{n}$ in (12), there are 19 unique difference patterns, and $\boldsymbol{\Delta B}$ is calculated to be

$$\boldsymbol{\Delta B}^{\mathsf{T}} = \begin{bmatrix}
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & 0 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & 0 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & 0 & -1 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 0 & 0 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 0 & 0 & 0 & -1 \\
0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 0 & 0 & -1 & 0 & 1 \\
0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & 0 & 0 & 0 & -1 & -1 \\
0 & 0 & 0 & 1 & -1 & -1 & 0 & -1 & -1 & 0 & -1 & 0 & 0 \\
0 & 0 & 1 & -1 & -1 & 0 & -1 & -1 & 0 & 0 & -1 & -1 & 0 \\
0 & 1 & -1 & -1 & 0 & -1 & -1 & 0 & -1 & 0 & -1 & 0 & 0 \\
1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & -1 & -1 & 0 & -1 & -1 & 0 & -1 & -1 & 0 & -1 & 0 & -1
\end{bmatrix}. \quad (33)$$

Using this enumerated form, (28) becomes

$$\overline{\mathrm{DNL}}_k = (\boldsymbol{\Delta b}_k^{\mathsf{T}} - 2^{-N} \cdot \mathbf{1}_M^{\mathsf{T}}) \cdot \boldsymbol{\Delta c}/C_u. \quad (34)$$

We define γ as the *number of unique difference patterns*, so the set of all $\overline{\mathrm{DNL}}_k$ values is given by

$$\overline{\mathbf{DNL}} = (\boldsymbol{\Delta B}^{\mathsf{T}} - 2^{-N} \cdot \mathbf{1}_\gamma \cdot \mathbf{1}_M^{\mathsf{T}}) \cdot \boldsymbol{\Delta c}/C_u. \quad (35)$$

V. Capacitor Placement

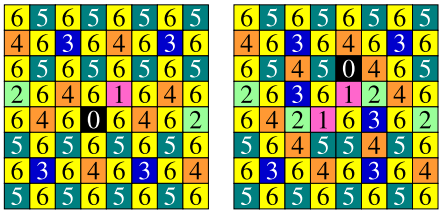
A. Sources of Variation

As explained in Section I, the three sources of variation investigated and modeled in this paper are the statistical ($\boldsymbol{\Delta c}^{\mathrm{sta}}$), systematic ($\boldsymbol{\Delta c}^{\mathrm{sys}}$), and parasitic ($\boldsymbol{\Delta c}^{\mathrm{par}}$) variations. The respective strategies applied to mitigate them are: maximizing unit dispersion, placement symmetry (e.g., adopting a common-centroid style), and unit clustering. Note that the mitigation techniques for $\boldsymbol{\Delta c}^{\mathrm{sta}}$ and $\boldsymbol{\Delta c}^{\mathrm{par}}$ oppose each other; this issue is addressed later in Section V-E.

B. Chessboard Placement Method

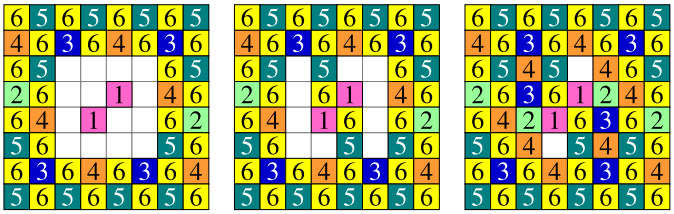
In [1], the analytical model served as the starting point to the derivation of a new "chessboard" placement method, concluding that in order to minimize the impact of random capacitor mismatch, σ^2_{DNL} is to be minimized. This paper only considered the placement, and as such, the objective was to minimize the impact of $\boldsymbol{\Delta c}^{\mathrm{sta}}$ and $\boldsymbol{\Delta c}^{\mathrm{sys}}$ on the DNL.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.



(a) $\hat{\boldsymbol{n}} = [32, 16, 8, 4, 2, 1, 1]$ (b) $\boldsymbol{n} = [26, 15, 10, 6, 4, 2, 1]$

Fig. 2. Chessboard placements for $N = 6$. (a) Binary [1]. (b) Nonbinary (Section V-C).



(a) (b) (c)

Fig. 3. Steps of the conversion process from a chessboard placement [Fig. 2(a)] for a binary capacitor ratio, into one for a nonbinary ratio [Fig. 2(b)].

The rules for common-centroid layout are followed when considering the design of the placement: coincidence, symmetry, dispersion, and compactness [17]. Coincidence and symmetry are obtained by orienting the axes of symmetry with respect to the center of the matrix. For the chessboard placement, all units obey these rules, except the two least significant capacitors $\hat{C}_1$ and $\hat{C}_0$, since they have odd multiplicities. However, both being 1, their impact is insignificant.

A depiction of the chessboard placement for $N = 6$ is given in Fig. 2(a). It can be seen from the alternating (chessboard) placement of the units, most prominently $\hat{C}_N$, that the level of dispersion is maximal, since individual unit capacitors are spread out as much as possible. Naturally, by using a square-shaped matrix, maximal compactness is achieved.

C. Modified Chessboard Placement for Nonbinary Ratios

The chessboard placement method is designed to fit a binary-weighted ratio, and hence is limited in this aspect. However, with the use of a nonbinary ratio, a pure chessboard placement (i.e., with maximal dispersion) can no longer be achieved. A generalization of the chessboard placement for nonbinary ratios is necessary, so a new placement method must be devised, that retains the qualities of the chessboard method, yet still accommodates the changes in the ratio. A chessboard placement for an arbitrarily chosen nonbinary ratio $\boldsymbol{n}$ is depicted in Fig. 2(b). Here, the total sum of $\boldsymbol{n}$ is equal to that of $\hat{\boldsymbol{n}}$, but the individual multiplicities have changed.

The steps of the new method are depicted in Fig. 3. From Fig. 2(a), excess unit capacitors are removed to form an empty square in the center, and C_1 is populated since its placement is fixed, giving Fig. 3(a). In order to form the square, too many unit capacitors have been removed; these are then repopulated from the center, giving Fig. 3(b). Incomplete unit capacitors are then populated in "local chessboards" within the empty positions, giving Fig. 3(c). Finally, the capacitors with odd multiplicities are spilled in the remaining empty positions, giving the final placement in Fig. 2(b).

D. Minimization of the INL and DNL

The main intention of the placement method is to minimize the nonlinearity metrics INL and DNL. From [1], any operation that modifies one affects the other in the same way, and so this objective can be reduced to only minimizing the DNL, since it has fewer values. From (20), because 2^{-N} is small, the expression is dominated by $\boldsymbol{\Delta}\hat{\boldsymbol{B}}$. Consider the (binary) case, where $k = 10$

$$\begin{aligned}\mathrm{DNL}_{10} &\overset{(20)}{=} (\boldsymbol{\Delta}\hat{\boldsymbol{b}}_{10}^{\mathsf{T}} - [2^{-N}, \ldots, 2^{-N}]) \cdot \boldsymbol{\Delta c}/C_u \\ &\overset{(19)}{\approx} [0, 0, 1, -1, \ldots, -1, 0]^{\mathsf{T}} \cdot \boldsymbol{\Delta c}/C_u \qquad (36) \\ &\propto [0, 0, 1, -1, \ldots, -1, 0]^{\mathsf{T}} \\ &\quad \cdot [\Delta C_{M-1}, \Delta C_{M-2}, \ldots, \Delta C_0] \\ &= \Delta C_{10} - \Delta C_9 - \Delta C_8 - \cdots - \Delta C_1. \qquad (37)\end{aligned}$$

The result from (37) tells us that the DNL is approximately proportional to the matrix product $\boldsymbol{\Delta}\hat{\boldsymbol{b}}_k^{\mathsf{T}} \cdot \boldsymbol{\Delta c}$. Since the goal is to minimize nonlinearity and hence reduce $|\,\mathrm{DNL}\,|$ as much as possible, the total variation for net k must equal the sum of all variations for nets with indices $< k$. Since the sum of the lower-indexed capacitor multiplicities is $n_k - 1$, the simplest way to perform matching is to ensure that the variations for all the affected (individual) units are identical. The approximation in (36) is only used to describe the matching process; for calculations, the complete definitions are used.

Qualitatively, this means that each row of $\boldsymbol{\Delta}\hat{\boldsymbol{B}}$ indicates the indices of the array capacitances that are to be matched with each other; ΔC for each array capacitor corresponding to a nonzero entry of $\boldsymbol{\Delta}\hat{\boldsymbol{b}}_k$ must be matched. This result is also valid for the nonbinary case. Since routing variations are a component of $\boldsymbol{\Delta c}$, this notion is also followed when routing.

The chessboard style fulfills the symmetry and dispersion criteria, adhering to the matching conditions such as (37), and hence is designed to minimize the worst-case $\mathrm{DNL}^{\mathrm{sta}}$ and $\mathrm{DNL}^{\mathrm{sys}}$, justified by the results presented in [1].

From (36), it can also be seen that reducing C_u results in increased sensitivity; larger mismatches between the components of $\boldsymbol{\Delta c}$ values would become amplified further.

E. Hybrid Placement

Since the chessboard placement offers the best (lowest) $\mathrm{DNL}^{\mathrm{sta}}$, it should be retained where possible. However, as identified earlier, the maximized dispersion of chessboard placement directly minimizes its routability, since unit capacitors are as separated from each other as possible.

In [1, Fig. 7], it was shown that across a number of placement styles, the most significant (MSB, highest-indexed) capacitors consistently influence the largest worst-case components of $\mathrm{DNL}^{\mathrm{sta}}$ ($3\sigma_{\mathrm{DNL}^{\mathrm{sta}}}$). In turn, the (unnecessary) dispersion of the least significant (LSB, lowest-indexed) capacitors can be sacrificed in order to improve their ease of routing.

The MSB capacitors clearly contribute the most to parasitics, primarily due to the amount of routing they require. However, its individual units are not affected as much as those of the less significant. For example, if a track contributes Δc_0 to both C_0 and C_N, C_0's units are affected by $\Delta c_0/n_0 = \Delta c_0$, while C_N's units are affected by $\Delta c_0/n_N \ll \Delta c_0$. In order to preserve the advantages that the dispersion brings without a significant sacrifice in routing variations, the MSB capacitors shall retain their chessboard placement.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

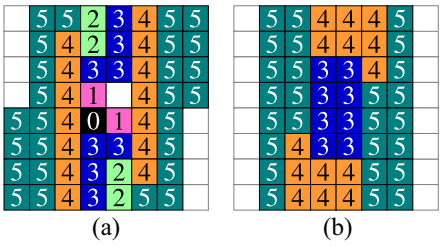


Fig. 4. (a) New grouped placement of size 8 × 8. (b) Modified grouped placement as described in Section V-F.

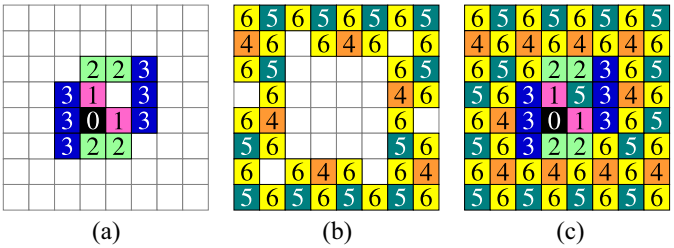


Fig. 5. Combining a (a) modified grouped placement of size 4, and a (b) chessboard placement with excess units removed, to form a (c) hybrid placement for $\boldsymbol{n}$ in Fig. 2(b). The modified C_X placement is omitted due to the smaller multiplicities in $\boldsymbol{n}$, where X has been determined to be 4. Note that the modifications suggested in Section V-F are omitted due to the smaller multiplicities in $\boldsymbol{n}$.

Targeting nonlinearity minimization, a new *hybrid* placement style was developed that balances the qualities of the chessboard style for the MSB capacitors (low DNL^{sta} and DNL^{sys}) with the reduction of routing complexity using a grouped style for the LSB capacitors. Use of a common-centroid placement means symmetry is ensured, so the DNL^{sys} is maintained to be as low as possible.

1) Grouped Placement: We first introduce a new placement style, shown in Fig. 4(a), that maintains the coincidence and symmetry qualities. Compactness is also evident from inspection. Dispersion, however, is the criterion that has been subject to change. Since there are two clusters of units for each array capacitor, this is the least dispersed a placement can be, while still retaining the other common-centroid qualities. Consequently, this placement has an *increased* DNL^{sta}.

Since the sum of the capacitor multiplicities does not fit exactly into a square, there are hence unpopulated positions; these are filled by other units during the placement process.

2) Construction: The hybrid placement is a square region of the same size as the chessboard placement. It consists of a square subregion in the center, placed in the grouped style, which is surrounded by capacitors placed in the chessboard style. First, the least significant array capacitor C_X *not* populated in the grouped style is determined. The size of the grouped subregion is the smallest square with even dimensions that fits *all* unit capacitors of the array capacitors from C_{X-1} to C_0. Excess spaces are filled by the units of more significant array capacitors. Hence, the choice of X directly affects the size of the grouped region, and vice versa. An analysis of $\overline{\text{DNL}}^{\text{sta}}$ in Section VII-A determines the selection of X.

The process outlined above and described in Fig. 5 refers to the nonbinary case. The conversion for the binary architecture is similar, but results in more clustered units.

After placing the grouped region [Fig. 5(a)] in the center of the chessboard, the multiplicities are corrected: any grouped units are removed from the outer region [Fig. 5(b)], and then in these newly removed positions the chessboard units are inserted, maintaining a chessboard-like structure, to form Fig. 5(c).

F. Modifications to Placement Based on Initial Routing

Two changes were made to the hybrid placement considering the results detailed in Section VII-A. These results are summarized here to complete the overview of the placement.

The grouped placement is modified so the units are moved toward the center. For example, starting from Fig. 4(a), C_3 would newly occupy the centermost region, as per Fig. 4(b).

The units of C_X are changed to form "rings" surrounding the grouped subregion. Excess units are then distributed along the two (vertically) centermost rows, forming *side branches*.

Finally, this modified grouped placement is combined into the chessboard placement and the affected chessboard capacitors are rearranged to produce the complete hybrid placement.

It was recognized that the array capacitors fall into four different categories of placement styles, and therefore, their unit capacitors would be routed correspondingly. Applying this categorization, we rewrite (11) as

$$\begin{aligned}\boldsymbol{c}_0^{\mathsf{T}} &= [\boldsymbol{c}_{\text{MSB}} \,|\, C_X \,|\, \boldsymbol{c}_{\text{group.}} \,|\, \boldsymbol{c}_{\text{LSB}}] \\ &= [C_N, \ldots, C_{X+1} \,|\, C_X \,|\, C_{X-1}, \ldots, C_3 \,|\, C_2, \ldots, C_0]. \quad (38)\end{aligned}$$

Based on the results in Section VII-A, the most feasible choice for the 12-bit architecture described by $\boldsymbol{n}$ in (12) is $X = 8 = N - 4$. From here, the (nominal) hybrid placement shall refer to this case. This corresponds to a hybrid placement with a size 14 grouped region for the nonbinary-weighted case; for the binary variant, this applies to a size 12 grouped region.

G. Algorithmic Implementation of the Placement Method

A PLACEMENT algorithm produces the placement, which itself executes the following subalgorithms: CHESSBOARD_NONBIN, GROUPED, and HYBRID.

For $N = 6$, $M = 7$, $\boldsymbol{n} = [26, 15, 10, 6, 4, 2, 1]$, and $X = 4$, the outputs of the above subalgorithms are depicted in Figs. 2(b), 5(a), and 5(c), respectively.

1) CHESSBOARD_NONBIN *(Nonbinary Chessboard):* Produces a chessboard-like placement $\boldsymbol{P}$ for the nonbinary capacitor ratio specified in $\boldsymbol{n}$, and is not required for the binary case. It is described in Algorithm 1.

2) GROUPED *(Grouped Subregion):* Produces a grouped-style placement $\boldsymbol{G}$ given X. Since this placement is bottom left-top right symmetric, all written steps are for the left half of the subregion. Any placements are also diagonally duplicated for the region below the axis of symmetry, i.e., where $\boldsymbol{P}[i][j] = k$, $\boldsymbol{P}[\dim - 1 - i][\dim - 1 - j] = k$ is repeated implicitly. The GROUPED algorithm is simple enough that it will be described textually.

First, a matrix $\boldsymbol{G}$ for the grouped region is created. C_3 is distributed in the two centermost columns of $\boldsymbol{G}$. Population starts from the top of the center-left column and "snakes" toward the bottom left corner: it continues downward, and when the edge is reached, it moves one position to the left and continues upward, and so on. When population of $\boldsymbol{c}_{\text{group.}}$ and $\boldsymbol{c}_{\text{LSB}}$ is complete, empty margins on all four sides are added. C_X units are then populated in these empty positions.

3) HYBRID *(Hybrid Placement):* Produces a hybrid placement $\boldsymbol{H}$ and is described in Algorithm 2. For the binary ratio, it

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Chessboard-style placement generator for the nonbinary architecture

$\boldsymbol{P}$ = CHESSBOARD_NONBIN(N, $\boldsymbol{n}$, $\hat{\boldsymbol{n}}$):
$\boldsymbol{P}$ = CHESSBOARD(N) ▷ Generate pure chessboard
for $k \coloneqq N : 3$ **step** -1 **where** $\hat{\boldsymbol{n}}(k) > \boldsymbol{n}(k)$
▷ Remove all units of k in a square from the center with side dimension $d_1 = 2\lfloor\sqrt{\hat{\boldsymbol{n}}(k) - \boldsymbol{n}(k)}/2\rfloor$
▷ Remove every second unit of k from the center until the edges of the square with side dimension $d_2 = 2\sqrt{2j'}$, where j' is the largest net where $\hat{\boldsymbol{n}}(j) < \boldsymbol{n}(j)$
▷ If too many were removed, repopulate back by spiralling out from center, only placing where they were originally removed from.
while *population incomplete*
▷ Generate a smaller chessboard based on number of unpopulated positions, starting at j'. If j' is fully populated, generate using $j' - 1$, and so on.
▷ Interleave the smaller chessboard in $\boldsymbol{P}$; place only in empty positions. If ratio reached, populate the next largest and valid unit (no units next to another of the same unit).

Algorithm 2 Combines $\boldsymbol{P}$ and $\boldsymbol{G}$ by placing $\boldsymbol{G}$ in the center of $\boldsymbol{P}$ and correcting the capacitor

$\boldsymbol{P}$ = HYBRID(N, $\boldsymbol{n}$, $\boldsymbol{P}$, $\boldsymbol{G}$, X):
▷ Remove all values $<= X$
▷ Populate $\boldsymbol{c}_{LSB}$ since they are fixed
▷ Insert $\boldsymbol{G}$ in the center of $\boldsymbol{P}$
▷ Populate C_X in the empty spaces of the two rows at $dim/2 - 1$ and $dim/2$, $dim = \sqrt{2^N}$
▷ Replace all missing C_N back to where they were removed from in the original chessboard placement.
▷ Populate remaining capacitors, i.e., $\boldsymbol{c}_{MSB} \setminus C_N$. All steps are for the top half of the matrix. Any placements from here are duplicated for the bottom half, i.e. where $\boldsymbol{P}[i][j] = k$, $\boldsymbol{P}[dim - 1 - i][dim - 1 - j] = k$ is repeated implicitly.
for $k \coloneqq N - 1 : X + 1$ **step of** -1
$\delta_0 = \delta = \boldsymbol{n}[k] -$ COUNT($\boldsymbol{P} == k$)
▷ Only consider rows with empty positions; the specific rows with empty positions will change as certain iterations fill their empty positions. Place only at empty places, then $\delta -= 1$.
while $\delta > 0$
if k *mod* $2 == 1$ ▷ Odd capacitors
▷ Place at next odd position only on odd rows
else ▷ Even capacitors
if $(\delta_0 - \delta)$ *mod* $2 == 0$ ▷ Even iterations
▷ Place at next position on odd rows
else ▷ Odd iterations
▷ Place at next even position on even rows
▷ If there are leftovers, populate at next position, searching across rows, starting from the top.

combines the output of GROUPED with the original chessboard placement. For the nonbinary ratio, it combines the outputs of the two previous subalgorithms.

VI. PARASITIC MATCHING-BASED ROUTING METHOD

The routing is separated into two phases: the first must be run once for minimal (but complete) connectivity; and the second is repeatedly run, but only after a parasitic extraction is performed, in order to iteratively adjust parasitics using parameters extracted from the preceding routing pass.

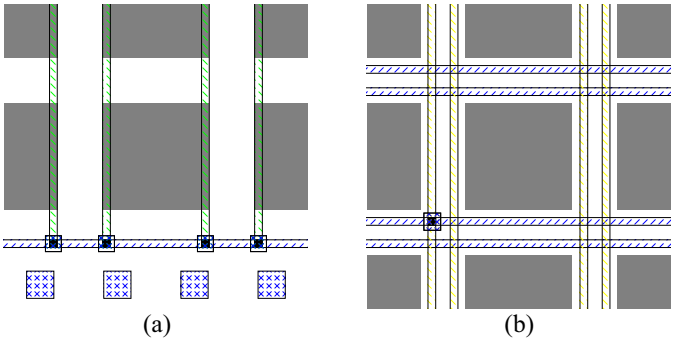


Fig. 6. Depictions of routing methodology. (a) Top plate routing and exit pins. (b) Routing channels within the gaps between the unit capacitors.

A. Routing (Nominal)

1) Top Plate: Since all top plates are connected together, routing is trivial and dependent on the geometry of the unit capacitor. A main (horizontal) trunk is constructed along the bottom of the array, with (vertical) subtrunks connecting all unit capacitors in one column to the main trunk. Visually this resembles a comb, as depicted in Fig. 6(a).

For the bottom plate routing, subtrunks follow the style shown in Fig. 6(b). In order to balance area whilst ensuring adequate connectivity, two routing channels were deemed necessary. Hence, there is sufficient spacing between unit capacitors to allow this. The comb structure from the top plate routing is retained. Finally, a via is depicted for interlayer connections, and connections from the subtrunks to the unit capacitors are omitted since they are technology-specific.

2) $\boldsymbol{c}_{MSB}$: The proposed matched routing style for $\boldsymbol{c}_{\mathrm{MSB}}$ can accommodate up to four nets. Since the routing technique is a parasitic mismatch-driven routing technique, this style is retained even for smaller N. The categorization from (38) can be appended with the condition $C_X = C_{N-4}$, but will retain its original definition to remain generic.

Routing for these nets is implemented with a horizontal trunk across the top of the matrix. Vertical subtrunks run to the bottom of the matrix, in the channels of every gap. In odd-indexed gaps, the trunks are connected to nets C_N and C_{N-1}. In even-indexed gaps, they are connected to the other two nets.

3) C_X: For C_X, the units surrounding the grouped region form a ring and hence are connected to each other directly. The side branches are then connected to the outermost ring, before the trunk is finally brought to the opposing outer edge, then down and across to the exit pin.

4) $\boldsymbol{c}_{group.}$: From opposite corners, a tree is constructed amongst units of the same net, always with the smallest routing distance. This routing is mirrored across the two halves.

Connections between unit capacitors and trunks, or other unit capacitors (in the case of the grouped capacitors), are naturally dependent on the geometry of the unit capacitor itself. The routing of the grouped capacitors hence is also dependent on this, as it primarily involves direct connections.

For the smallest two capacitors in this category, their placements within the very center of the matrix cause them to be highly susceptible to parasitics. Their routing is hence

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

carefully controlled by bringing them to opposite outer edges, then following that edge down and across to the exit pin.

For the other capacitors, their connections to the corresponding exit pins are performed with the shortest path.

5) $\boldsymbol{c}_{LSB}$: Due to the small number of unit capacitors and their intended placements at the edge of the matrix, these are routed directly to the exit pins with the shortest paths.

B. Routing (Parasitic Compensation)

With the first pass of routing completed, parasitic extraction is performed to identify the parasitic capacitances between the top and bottom plates of each array capacitor. Extracted values from the first pass are used to perform a comparison across all nets. These results are used as the starting point for further, repeated passes of routing.

The aim of the iterated routing process is to adjust the lengths of the routing and hence the coupled parasitics between its routing tracks, and either the top plate or *VSS*, in order to match the parasitics affecting every array capacitor.

Since the nets of $\boldsymbol{c}_{\mathrm{MSB}}$ are the most sensitive and hence should be prioritized in the matching process, they are analyzed first in order to reduce the number of iterations. For these nets, parasitics are added by copying their vertical trunks on a different metal layer. Reduction of parasitics is achieved by adding parallel tracks connected to *VSS*.

Although routing of the trees within the grouped regions is not subjected to large parasitics, the routing that connects these regions to the exit pins is vulnerable to the parasitics from the more complex routing. To resolve this, the exit routing for $\boldsymbol{c}_{\mathrm{MSB}}$ is purposely laid close to the substrate.

Algorithm 3 Deterministically route all capacitors to achieve bare minimum connectivity

$\boldsymbol{L}$ = ROUTE($\boldsymbol{P}$):
▷ Base trunk from left to right edge below bottom edge
for $i \coloneqq 0 : dim - 1$ ▷ Route top plate
▷ Subtrunks both side of column, base trunk → top
▷ Connect from base trunk to exit pin
for $k \coloneqq N : C_{N-3}$ ▷ Route $\boldsymbol{c}_{MSB}$
▷ Base trunk from left to right edge above top edge
for $i \coloneqq 0 : dim - 2$ **step of** 2
if $i \ mod \ 2 == 0$ ▷ Even: N, $N-1$
▷ Subtrunk right channel of column $i \rightarrow N$
▷ Subtrunk left channel of column $i+1 \rightarrow N-1$
else if $i \ mod \ 2 == 1$ ▷ Odd: $N-2$, $N-3$
▷ Subtrunk right channel of column $i \rightarrow N-2$
▷ Subtrunk left channel of column $i+1 \rightarrow N-3$
▷ Connect rightmost subtrunk to exit pin k
for $k \coloneqq X : 3$ ▷ Route C_X and $\boldsymbol{c}_{group.}$
▷ Perform on left half first, then repeat for right half
▷ Connect units of k on bottom row horizontally
▷ Connect units of k down each column vertically
if $k == X$
▷ Connect units on side regions, then → exit pin X
else if $5 \leq k \leq X-1$
▷ Direct exit along channel closest to exit pin k
else if $3 \leq k \leq 4$
▷ Go to side edge (left for 3, right for 4), down the edge to the row of exit pins, go across to exit pin k
▷ Route $\boldsymbol{c}_{LSB}$ direct → exit pins

C. Algorithmic Implementation of the Routing Method

Routing is performed to achieve the layouts shown in Fig. 6. Both top plate and $\boldsymbol{c}_{\mathrm{MSB}}$ routing styles follow the style of Fig. 6(a). Algorithm 3 describes the algorithm for nominal routing, necessary for complete connectivity and hence basic working functionality, Algorithm 4 depicts how the parasitics are parametrically controlled, and Algorithm 5 depicts the complete place and route procedure.

VII. Experimental Results

The experimental results were produced using commercial tools. The abstract placement generation was done in MATLAB. The actual circuit synthesis was performed using Virtuoso Layout Suite L User Guide Product Version IC6.1.7. The parasitics were extracted with Cadence Quantus QRC Extraction 18.1.

A. Selection and Feasibility of Placements

In order to verify that the hybrid placement can be justifiably used instead of the chessboard placement, the changes in $\overline{\mathrm{DNL}}^{\mathrm{sta}}$ and $\boldsymbol{\Delta c}^{\mathrm{par}}$ were evaluated against the chessboard placement, shown in Fig. 7. Results for $\boldsymbol{\Delta c}^{\mathrm{sys}}$ are omitted due to their insignificance compared to the other sources. Unit capacitors with dimensions of $2.35 \times 2.35\ \mu^2$ and capacitance $C_u = 12.913$ fF were used. The worst-case value ($3\,\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}}$, as defined in [4]) and its components were calculated for varying sizes of the grouped region. In all cases, both horizontal and vertical spacing between unit capacitors were fixed at $1\ \mu$.

1) $\overline{DNL}^{sta}$: Due to the statistical nature of these variations, the $3\,\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}}$ values are evaluated for each component of the $\overline{\mathrm{DNL}}^{\mathrm{sta}}$, sweeping the size of the grouped region. These components refer to each of the γ elements of the vector that results from the evaluation of (35). Two trends are clearly visible from Fig. 7. First, the $\overline{\mathrm{DNL}}$ components with higher indices tend to have increased values; these components correspond with the MSB capacitors. Second, $\max(3\,\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}})$ also increases with larger grouped regions. This is consistent with what was expected, since a larger grouped region translates to a smaller chessboard region.

To choose the size of the grouped region, there is a tradeoff between routing complexity (and hence, parasitics) and nonlinearity. Smaller grouped region sizes (i.e., 4, 6, and 8) offer no significant improvement with regards to ease of routing; if the grouped region is too large, the chessboard quality is lost.

In general, a value would be defined for the maximum amount a $\max(3\,\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}})$ for a given size is allowed to worsen that of the size 0 case. The largest X that is still in compliance would be the one that is selected. For $N = 12$, a grouped region of size 14 ($X = 8$) best balances routing complexity with an acceptable sacrifice in $\overline{\mathrm{DNL}}^{\mathrm{sta}}$, and hence shall be selected for further tests of routing benchmarks. This placement has a $\max(3\,\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}})$ value of $0.955V_{\mathrm{LSB}}$, an acceptable increase of $0.169V_{\mathrm{LSB}}$ from the $0.786V_{\mathrm{LSB}}$ of the chessboard (size 0 grouped region). As another point of reference, the grouped placement of size 26 ($X = 10$) was also considered.

2) $\boldsymbol{\Delta c}^{par}$: Based on the results of the previous step, certain sizes of the grouped region are excluded. Routing for nets other than $\boldsymbol{c}_{\mathrm{LSB}}$ and $\boldsymbol{c}_X$ was performed using ROUTE from

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4 Based on Factors for Each Net, Control the Amount of Parasitics on That Net

PARASITIC_COMPENSATION($\boldsymbol{L}$, N, $\boldsymbol{f}$):
▷ Remove any compensation-related routing
for $k := 0 : 2$ ▷ $\boldsymbol{c}_{LSB}$
▷ $\boldsymbol{f}[k]$ controls distance between routing to V_X trunk
for $k := 3 : X - 1$ ▷ $\boldsymbol{c}_{group.}$
if $\boldsymbol{f}[k] < 0.0$
▷ *VSS* on either side of vertical trunk, length $\sim \boldsymbol{f}[k]$
else if $\boldsymbol{f}[k] > 0.0$
▷ V_X on either side of vertical trunk, length $\sim \boldsymbol{f}[k]$
if $\boldsymbol{f}[X] < 0.0$ ▷ C_X
▷ *VSS* on either side of side units, length $\sim \boldsymbol{f}[X]$
else if $\boldsymbol{f}[X] > 0.0$
▷ V_X on either side of side units, length $\sim \boldsymbol{f}[X]$
for $k := X + 1 : N$ ▷ $\boldsymbol{c}_{MSB}$
▷ Duplicate nominal routing on the next vertical layer. Subtrunks are populated inwards from the outer edges. The number of subtrunks is controlled by $\boldsymbol{f}[k]$, such that the final subtrunk may not span the full height of the matrix.

Algorithm 5 Full Algorithm of Placement, Routing, and Parasitic Compensation

PASTEL(N, M, $\boldsymbol{n}$, X):
$\boldsymbol{P}$ = PLACEMENT($N, M, \boldsymbol{n}, X$) ▷ Subsection V-G
▷ Instantiate unit capacitors based on $\boldsymbol{P}$ onto layout $\boldsymbol{L}$
threshold = 0.001, *compliant* = False, *iterations* = 0
$\delta_{max} = 0$, $\boldsymbol{f} = [0, \ldots, 0]$ ▷ Initialize to M 0s
while *compliant* == False **and** *iterations* < *max_iterations*
$\boldsymbol{L}$ = ROUTE($\boldsymbol{P}$) ▷ Or re-route, from Algorithm 3
$\boldsymbol{p}$ = EXTRACT_PARASITICS() ▷ Parasitics for each net
▷ Generate new factors for parasitic compensation
▷ Adjust elements of $\boldsymbol{f}$ based on impact on *DNL*
PARASITIC_COMPENSATION($N, \boldsymbol{f}$) ▷ Algorithm 4
for $n := 0 : N - 1$ ▷ Compare every difference
for $m := n + 1 : N$
if $|p[n] - p[m]| > \delta_{max}$ **then** $\delta_{max} = |p[n] - p[m]|$
compliant = δ_{max} < *threshold*, *iterations* ++

Algorithm 3. Routing for C_X adheres to that for $\boldsymbol{c}_{MSB}$, and $\boldsymbol{c}_{\mathrm{LSB}}$ to $\boldsymbol{c}_{\mathrm{group.}}$, respectively. Parasitic extraction was then performed, and $\boldsymbol{\Delta c}^{\mathrm{par}}$ used for comparison. Fig. 8 depicts these results from the routing of the selected placements.

The significant parasitics on $\boldsymbol{c}_{\mathrm{LSB}}$ are clearly noticeable. For grouped regions of sizes 14 and 26, X is 8 and 10, respectively. It can be seen that in both cases, there is a spike in the curve at each of the corresponding C_X values compared to $C_{X\pm1}$, creating an undesirable mismatch in parasitics. The changes in Section V-F were devised to rectify these problems.

In accordance with the theory, the parasitics for the chessboard placement are clearly higher than that of the hybrid placements, as shown by the results of Fig. 8. Not only do they have lower total parasitics, but the "unit parasitics" across each net are also already more consistent (neglecting $\boldsymbol{c}_{\mathrm{LSB}}$), simplifying the matching process, which is based on (37).

The parasitic performance of the size 26 did not offer a significant improvement over that of the size 14. For most of the nets, the parasitic values for both placements were almost identical. Regardless, any such mismatches can still be controlled by the compensation phase of the routing.

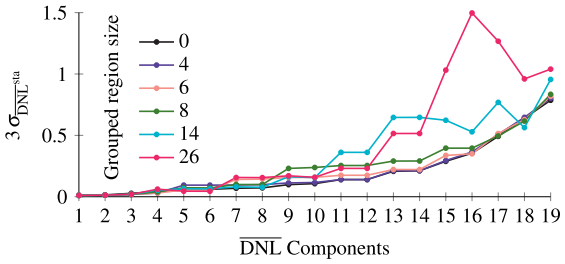


Fig. 7. Components of $3\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}}$ across increasing grouped region size (indicated by the legend).

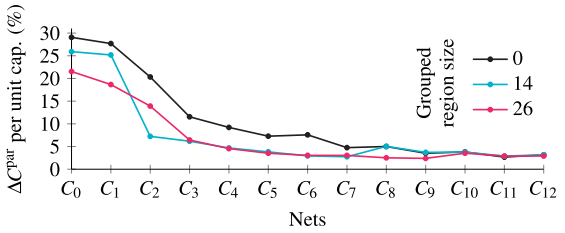


Fig. 8. Comparison between the extracted parasitics per unit capacitor of each net, after routing.

The final hybrid placement for the 12-bit architecture described by $\boldsymbol{n}$ in (12) has a size 14 grouped region, includes the placement changes to C_X and $\boldsymbol{c}_{\mathrm{LSB}}$, and is depicted in Fig. 9.

The unit capacitors are indexed in the right half of the figure. The left half depicts the routing styles of the different nets. To achieve matching, the MSB capacitors have two vertical trunks within one gap [as per Fig. 6(b)]. The pairs that are closer together represent the two channels within a single gap. These pairs alternate through the gaps in order to cover all units.

The trees formed to connect the grouped capacitors, as per Algorithm 3, are also shown. Connections to the exit pins are omitted due to its simplicity: simply route along the unused vertical channel closest to the corresponding exit pin.

B. Nonlinearity Analysis and Comparison

1) Advantages of the Hybrid Placement: The results presented in Section VII-A are only valid for the grouped and hybrid placements without the changes detailed in Section V-F, since these results were used to establish the validity of the proposed placements by comparing them to the chessboard placement. From here, all presented results belong to the placement with the modifications from Section V-F ($\boldsymbol{c}_{\mathrm{LSB}}$ moved to the matrix edge, and redistribution of C_X in rings), for both 12- and 10-bit hybrid placements, implicitly repeating the previous tests in order to ensure that the improvements are still retained.

First, we evaluate the worst-case $\overline{\mathrm{DNL}}^{\mathrm{sta}}$ of the hybrid placement compared to the chessboard (most dispersed), and the grouped (least dispersed) placements, depicted in Fig. 10. The worst-case value for the hybrid is $1.483V_{\mathrm{LSB}}$, which is larger than that of the chessboard ($1.151V_{\mathrm{LSB}}$) by $0.332V_{\mathrm{LSB}}$, an acceptable value. The grouped placement has a significantly larger worst-case value, $5.859V_{\mathrm{LSB}}$. Since its placement already suffers from such a significant $\overline{\mathrm{DNL}}^{\mathrm{sta}}$, its routing will not be considered, in favor of the other placements.

Using the first-order model from [1] for systematic variations, the $|\overline{\mathrm{DNL}}^{\mathrm{sys}}|_{\max}$ for these placements were all evaluated

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

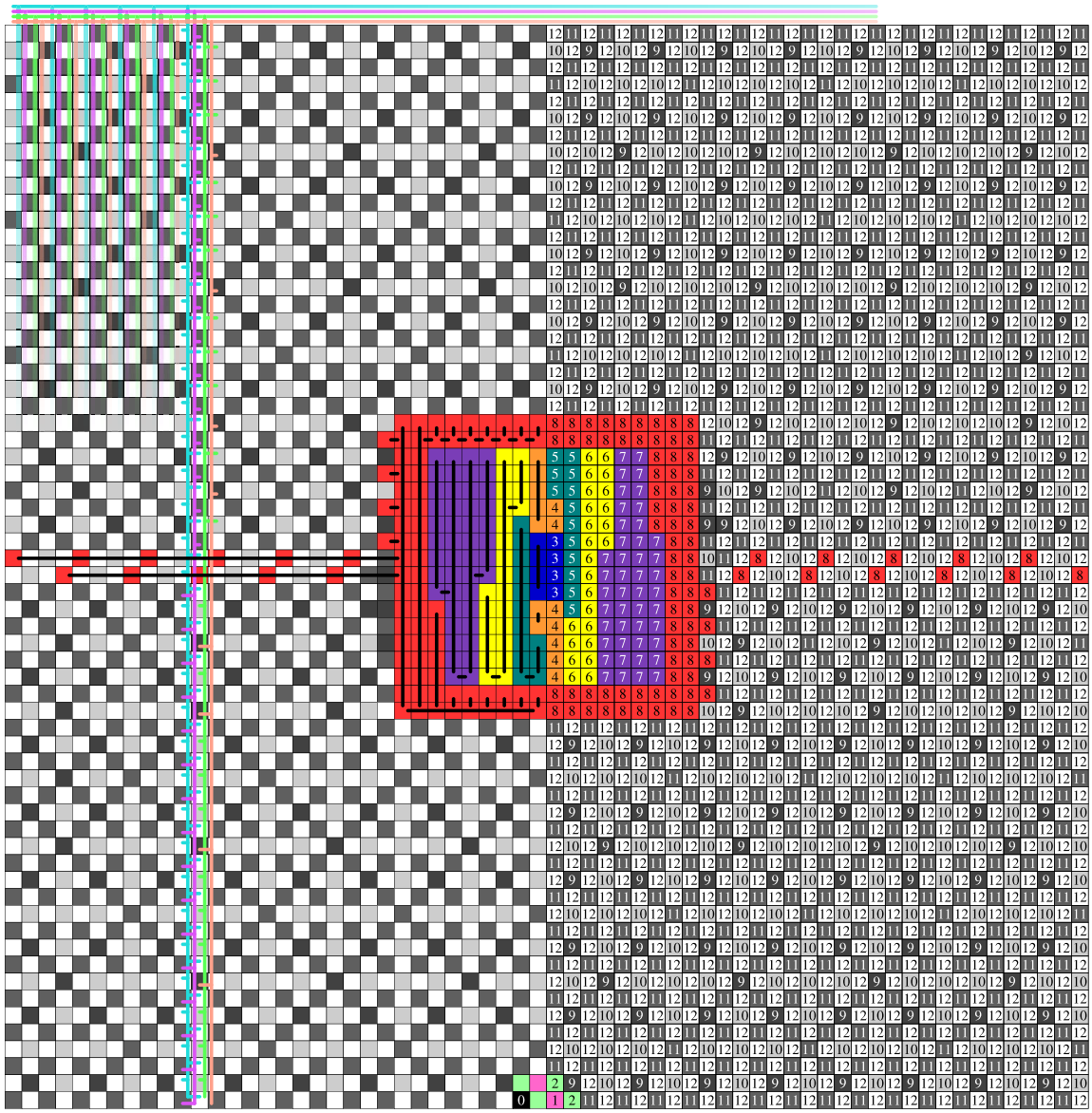


Fig. 9. Final hybrid placement with a grouped region of size 14, for the 12-bit nonbinary architecture characterized by (12), also depicting routing.

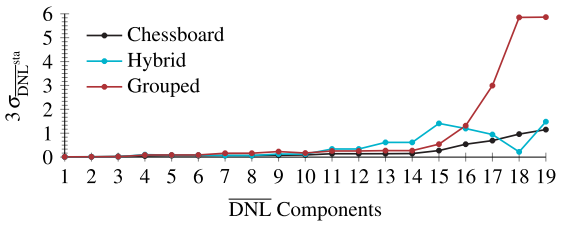


Fig. 10. Components of $3\,\sigma_{\overline{\mathrm{DNL}}^{\mathrm{sta}}}$ across three placement styles of varying dispersion.

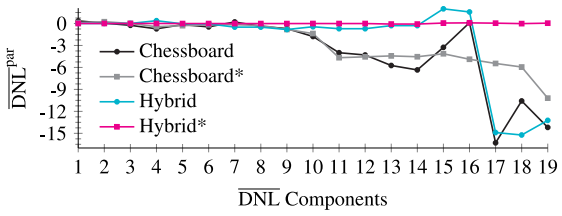


Fig. 11. Components of $\overline{\mathrm{DNL}}^{\mathrm{par}}$ for the chessboard and hybrid placements before, and after (*) compensation.

to be < 0.001, which is much less than the other variation sources. Since the common-centroid placement rules were followed, the impact due to $\boldsymbol{\Delta c}^{\mathrm{sys}}$ has been minimized, as explained in [22], and experimentally justified in [1].

2) Compensation: The $\overline{\mathrm{DNL}}^{\mathrm{par}}$ is evaluated next. From Fig. 11, the results before compensation for both placements show $|\,\overline{\mathrm{DNL}}^{\mathrm{par}}\,|_{\max} > 15$. With compensation, the $|\,\overline{\mathrm{DNL}}^{\mathrm{par}}\,|_{\max}$ for the chessboard could be reduced; however, it still greatly violates thresholds defined by Lin *et al.* [2] for $|\,\mathrm{DNL}\,|$, which should be < 0.5 in order to have no missing codes. The hybrid placement provides the ability to reduce $|\,\overline{\mathrm{DNL}}^{\mathrm{par}}\,|_{\max}$ to a value < 0.1.

PASTEL drives the $|\,\overline{\mathrm{DNL}}^{\mathrm{par}}\,|_{\max}$ to be as low as possible without the parasitics becoming excessively large, or when an iteration limit is reached. From the nominal routing

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ROUTING RESULTS OF PASTEL'S HYBRID PLACEMENTS BEFORE AND AFTER COMPENSATION, USING COMPLETE DEFINITIONS OF INL AND DNL

Placement	Compensation	$\lvert \mathrm{INL}^{\mathrm{par}} \rvert_{\max}$	$\lvert \mathrm{DNL}^{\mathrm{par}} \rvert_{\max}$
Nonbinary 12b	Before	12.536	15.226
	After	0.083	0.083
Binary 12b	Before	11.156	16.005
	After	0.105	0.103
Binary 10b	Before	4.433	5.638
	After	0.099	0.105

TABLE IV
ROUTING RESULTS OF PASTEL'S 10-BIT BINARY HYBRID PLACEMENT IN COMPARISON WITH LIN *et al.*'S [2] AND HUANG *et al.*'S [5] APPROACHES FOR 10-BIT ARCHITECTURES

Method	C_u (fF)	Area (μm^2)	$\lvert \mathrm{INL} \rvert^*$	$\lvert \mathrm{DNL} \rvert^*$
PASTEL	12.913	11,278.44	0.099	0.105
Lin et al. [2]	82.000	244,877.96	0.241	0.180
Huang et al. [5]	32.700	*Not given*	0.370	0.430

with ROUTE (before compensation), it can be seen that the maximum values of $\lvert \overline{\mathrm{DNL}}^{\mathrm{par}} \rvert$ for both placements are > 0.5.

As the iterations progress, values typically reduce to approximately $\lvert \overline{\mathrm{DNL}}^{\mathrm{par}} \rvert_{\max} = 0.1$, whilst further iterations do not change the value significantly. These results were obtained for the linearized definitions. When applying the complete definitions, very comparable results were obtained, which justifies the use of the linear model.

Demonstrating the functionality of PASTEL, these routing results for a selection of placements are collated in Table III. Other nonbinary architectures were omitted due to the complex task of generating new capacitor ratios altogether. Due to their close mathematical relationship, $\lvert \mathrm{INL}^{\mathrm{par}} \rvert_{\max}$ also indirectly converges to be approximately 0.1. In all cases, the $\lvert \mathrm{INL}^{\mathrm{par}} \rvert$ and $\lvert \mathrm{DNL}^{\mathrm{par}} \rvert$ were greatly reduced to be well within 0.5.

C. Comparison With the State-of-the-Art

The results for the 10-bit binary architecture PASTEL were used for the purposes of comparison against the results from Lin *et al.* [2] and Huang *et al.* [5], who presented results for up to 10- and 11-bit architectures, respectively, in contrast to PASTEL, which is designed for up to 12-bit architectures.

According to the results presented in Table IV, PASTEL was superior with regards to C_u, and total area, achieving smaller (better) values compared to [2] and [5], as well as a number of older works [9], [12], [13], where the comparisons were performed in [2, Sec. V].

The nonlinearity metrics in Table IV are starred, as the exact definitions presented vary across different works. Post-layout parasitic extractions are always included in all sets of results, however, whilst PASTEL and Lin *et al.* [2] considered the INL and DNL from the largest components, RMS values across all components are presented in [5], with maximum values only provided for a 9-bit architecture. Nevertheless, amongst the suitably chosen nonlinearity metrics for comparison, PASTEL still demonstrates desirable values.

Considering routing specifically, Ho *et al.* [12] aimed to stringently match capacitance multiplicities, which would still implicitly result in minimized nonlinearity when the analytical models of nonlinearity presented in this paper are applied. However, the less significant a capacitor is, the smaller its impact, and hence the matching for these nets can actually be ignored to a certain extent, provided that the matching of the MSB nets is performed correctly. The routing in PASTEL is simplified such that the MSB nets are prioritized, and lengths are adjusted toward the macro goal of minimizing INL and DNL.

VIII. CONCLUSION

In this paper, the analytical model from [1] was generalized to additionally account for nonbinary-weighted capacitor arrays. The chessboard style, also introduced in [1], was first extended to consider the nonbinary case, then improved to account for routing-based variations. A sequence of design steps led to the development of the hybrid placement style that combined the chessboard style from [1] with a new grouped style, inspired by and extended from [2].

The hybrid style was then augmented by a complementary routing technique simplified from that presented in [12]. The routing applies different techniques to connect the capacitors placed in different styles, with particular emphasis on matching routing parasitics based on the generalized analytical model. A new compensation technique is used to adjust the parasitics of each net based on the analysis of the parasitic extraction performed on the previous routing operation. The compensation repeats iteratively until a threshold is achieved, or a certain maximum number of iterations is exhausted.

The complete design workflow ("PASTEL") that combined the new placement and routing techniques, targets the minimization of variations across all sources that were modeled in this paper, and hence minimization of the INL and DNL.

Through comparisons with other state-of-the-art works, PASTEL demonstrated superiority in several benchmark criteria, including area, total capacitance, INL, and DNL.

REFERENCES

[1] F. Burcea, H. Habal, and H. E. Graeb, "A new chessboard placement and sizing method for capacitors in a charge-scaling DAC by worst-case analysis of nonlinearity," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 9, pp. 1397–1410, Sep. 2016.

[2] M. P.-H. Lin, V. W.-H. Hsiao, C.-Y. Lin, and N.-C. Chen, "Parasitic-aware common-centroid binary-weighted capacitor layout generation integrating placement, routing, and unit capacitor sizing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 36, no. 8, pp. 1274–1286, Aug. 2017.

[3] T. Ogawa *et al.*, "SAR ADC algorithm with redundancy," in *Proc. IEEE Asia–Pac. Conf. Circuits Syst. (APCCAS)*, Nov. 2008, pp. 268–271.

[4] F. Burcea, H. Habal, and H. Graeb, "Procedural capacitor placement in differential charge-scaling converters by nonlinearity analysis," in *Proc. 53rd ACM/EDAC/IEEE DAC*, Austin, TX, USA, 2016, pp. 1–6.

[5] C.-C. Huang, J.-E. Chen, and C.-L. Wey, "PACES: A partition-centering-based symmetry placement for binary-weighted unit capacitor arrays," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 36, no. 1, pp. 134–145, Jan. 2017.

[6] J. Shen *et al.*, "A 16-bit 16-MS/s SAR ADC with on-chip calibration in 55-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 1149–1160, Apr. 2018.

[7] W. Guo, Y. Kim, A. H. Tewfik, and N. Sun, "A fully passive compressive sensing SAR ADC for low-power wireless sensors," *IEEE J. Solid-State Circuits*, vol. 52, no. 8, pp. 2154–2167, Aug. 2017.

[8] M. Zhang, K. Noh, X. Fan, and E. Sánchez-Sinencio, "A 0.8–1.2 V 10–50 MS/s 13-bit subranging pipelined-SAR ADC using a temperature-insensitive time-based amplifier," *IEEE J. Solid-State Circuits*, vol. 52, no. 11, pp. 2991–3005, Nov. 2017.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.

[9] M. P.-H. Lin, Y.-T. He, V. W.-H. Hsiao, R.-G. Chang, and S.-Y. Lee, "Common-centroid capacitor layout generation considering device matching and parasitic minimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 7, pp. 991–1002, Jul. 2013.
[10] M. P.-H. Lin, V. W.-H. Hsiao, and C.-Y. Lin, "Parasitic-aware sizing and detailed routing for binary-weighted capacitors in charge-scaling DAC," in *Proc. 51st ACM/EDAC/IEEE DAC*, San Francisco, CA, USA, Jun. 2014, pp. 1–6.
[11] P.-Y. Chou, M. P.-H. Lin, and H. Graeb, "An integrated placement and routing for ratioed capacitor array based on ILP formulation," in *Proc. VLSI-DAT*, Apr. 2016, pp. 1–4.
[12] K.-H. Ho, H.-C. Ou, Y.-W. Chang, and H.-F. Tsao, "Coupling-aware length-ratio-matching routing for capacitor arrays in analog integrated circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 2, pp. 161–172, Feb. 2015.
[13] Y. Li, Z. Zhang, D. Chua, and Y. Lian, "Placement for binary-weighted capacitive array in SAR ADC using multiple weighting methods," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 33, no. 9, pp. 1277–1287, Sep. 2014.
[14] J.-E. Chen, P.-W. Luo, and C.-L. Wey, "Placement optimization for yield improvement of switched-capacitor analog integrated circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 29, no. 2, pp. 313–318, Feb. 2010.
[15] C.-W. Lin, J.-M. Lin, Y.-C. Chiu, C.-P. Huang, and S.-J. Chang, "Mismatch-aware common-centroid placement for arbitrary-ratio capacitor arrays considering dummy capacitors," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 12, pp. 1789–1802, Dec. 2012.
[16] C. F. T. Soares and A. Petraglia, "Automatic placement to improve capacitance matching using a generalized common-centroid layout and spatial correlation optimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 10, pp. 1691–1695, Oct. 2015.
[17] A. Hastings, *The Art of Analog Layout*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2006.
[18] T. C. Carusone, D. A. Johns, and K. W. Martin, *Analog Integrated Circuit Design*. Hoboken, NJ, USA: Wiley, 2012.
[19] J. L. McCreary and P. R. Gray, "All-MOS charge redistribution analog-to-digital conversion techniques—I," *IEEE J. Solid-State Circuits*, vol. SSC-10, no. 6, pp. 371–379, Dec. 1975.
[20] P. Harpe, E. Cantatore, and A. van Roermund, "A 10b/12b 40 kS/s SAR ADC with data-driven noise reduction achieving up to 10.1b ENOB at 2.2 fJ/conversion-step," *IEEE J. Solid-State Circuits*, vol. 48, no. 12, pp. 3011–3018, Dec. 2013.
[21] T. Ogawa *et al.*, "Non-binary SAR ADC with digital error correction for low power applications," in *Proc. IEEE Asia–Pac. Conf. Circuits Syst.*, Dec. 2010, pp. 196–199.
[22] C. C. McAndrew, "Layout symmetries: Quantification and application to cancel nonlinear process gradients," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 36, no. 1, pp. 1–14, Jan. 2017.



Ye X. Ding received the B.E. (Hons.) degree in electrical engineering from the University of Queensland, Brisbane, QLD, Australia, in 2014, and the M.Sc. degree in communications engineering from the Technical University of Munich, Munich, Germany, in 2018.

From 2014 to 2016, he was a Digital IP Design Engineer with Nano Silicon, Pty., Ltd., Brisbane. He is currently a Circuit Design Engineer of Embedded Memories and e-Fuses with Infineon Technologies AG, Neubiberg, Germany.

Mr. Ding was a recipient of the Second Place at the 2018 International Microelectronics Olympiad of Armenia (in co-operation with IEEE), and the Analog Domain Cadence Master Thesis Award in 2019.



Florin Burcea received the B.Sc. degree in electronics and telecommunication engineering from the University Politehnica of Bucharest, Bucharest, Romania, in 2013, and the M.Sc. degree in communications engineering from the Technical University of Munich, Munich, Germany, in 2015, where he is currently pursuing the Ph.D. degree with the Chair of Electronic Design Automation, focusing on the design methodology and design automation of analog circuits.

Mr. Burcea was a recipient of the First Place in the Analog Integrated Circuits section of the Annual National Contest "Tudor Tănăsescu" in Romania, in 2013, the First Place in the Synopsys Annual International Microelectronics Olympiad of Armenia, from 2015 to 2017 (both contests are held in partnership with IEEE), the Award of the Association for Electrical, Electronic and Information Technologies (VDE) for his master thesis in 2016.



Husni Habal received the B.Sc. degree in computer engineering and the M.Sc. degree in electrical and computer engineering from Oregon State University, Corvallis, OR, USA, in 2001 and 2004, respectively, and the Dr.-Ing. degree in electrical engineering from the Technical University of Munich (TUM), Munich, Germany, in 2013.

Funded by the German Research Foundation, he held a post-doctoral position with the Institute for Electronic Design Automation, TUM, until 2016, with a focus on the modeling of transistor aging mechanisms for numerical simulation and analog circuit optimization. He is currently with Infineon Technologies AG, Neubiberg, Germany, focusing on the design flow, implementation, and verification methods of analog circuits. His current research interests include analog and mixed-signal design and synthesis automation.



Helmut E. Graeb (M'02–SM'03–F'14) received the Dipl.-Ing., Dr.-Ing., and Habilitation degrees in electrical engineering from the Technical University of Munich (TUM), Munich, Germany, in 1986, 1993, and 2008, respectively.

He was with Siemens Corporation, Munich, from 1986 to 1987, where he was involved in the design of DRAMs. Since 1987, he has been with the Chair of Electronic Design Automation, TUM, where he has been the Head of the Research Group since 1993. He has published around 190 papers. His current research interests include design automation for analog and mixed-signal circuits, with particular emphasis on Pareto optimization of analog circuits considering parameter tolerances, analog design for yield and reliability, hierarchical sizing of analog circuits, analog/mixed-signal test design, discrete sizing of analog circuits, structural analysis of analog and digital circuits, and analog layout.

Dr. Graeb was a recipient of the 2008 Prize of the Information Technology Society (ITG), the 2004 Best Teaching Award of the TUM EE Faculty Students Association, and the Third Prize of the 1996 Munich Business Plan Contest, and was nominated for best papers at the Design Automation Conference (DAC), the International Conference on Computer-Aided Design (ICCAD), and the Design, Automation and Test in Europe (DATE) conference. He has served as an Associate Editor for the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—PART II: ANALOG AND DIGITAL SIGNAL PROCESSING and the IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, the Vice-President Publications of the IEEE Council on Electronic Design Automation, an Executive Committee Member of the ICCAD, a member or the Chair of the Analog Program Subcommittees of the ICCAD, DAC, and DATE conferences, and a member of the Technical Advisory Board of MunEDA GmbH, Munich, which he co-founded. He is a member of VDE ITG and VDE/VDI-Society Microelectronics, Microsystems, and Precision Engineering.

Authorized licensed use limited to: Tamkang Univ.. Downloaded on February 04,2026 at 06:36:33 UTC from IEEE Xplore. Restrictions apply.